

PATA DE VELUDO / PAGAMENTOS

pagamentos — gateway extensível

Mercado Pago

Strategy Pattern

Pix · Cartão · Boleto

arquitetura

O sistema de pagamentos usa o **Strategy Pattern**: cada gateway é uma implementação de `PaymentGateway`. O `OrderService` nunca conhece o gateway — apenas chama a `factory`. Trocar ou adicionar um gateway é zero impacto no restante do sistema.

`PaymentGateway.java` – interface principal

```
// Interface (nunca muda quando adicionar novo gateway)
public interface PaymentGateway {
    PaymentResponse createPayment(PaymentRequest request);
    PaymentStatus getStatus(String externalId);
    RefundResponse refund(String externalId, BigDecimal amount);
    void processWebhook(String payload, String signature);
    String provider();
}
```

`PaymentGatewayFactory.java` – Spring injeta todos os gateways automaticamente

```
@Service
public class PaymentGatewayFactory {
    private final Map<String, PaymentGateway> gateways;

    public PaymentGatewayFactory(List<PaymentGateway> all) {
        this.gateways = all.stream()
            .collect(Collectors.toMap(
                PaymentGateway::provider,
                g -> g
            ));
    }

    public PaymentGateway get(String provider) {
        return Optional.ofNullable(gateways.get(provider))
    }
```

```

        .orElseThrow(() -> new UnsupportedOperationException(provider
    }
}

```

StripeGateway.java – adicionar Stripe amanhã: zero mudança em OrderService

```

// Adicionar Stripe amanhã: zero mudança em OrderService
@Component("stripe")
public class StripeGateway implements PaymentGateway {
    @Override public String provider() { return "stripe"; }
    // ... implementar os 4 métodos
}

```

mercado pago — atual

Gateway ativo em produção. Três métodos de pagamento suportados.



Pix

5% de desconto automático aplicado no PaymentRequest.



Cartão de Crédito

Parcelamento em até 4x sem juros configurado via preference.



Boleto

Vencimento em 3 dias úteis. URL do boleto retornada no response.

webhook events tratados

● payment.created	Pedido registrado, aguarda confirmação
● payment.approved	Pagamento aprovado — dispara fulfillment
● payment.rejected	Pagamento recusado — status do pedido atualizado
● payment.refunded	Reembolso processado pelo gateway

adicionar novo gateway

1

Criar a classe do gateway

Criar `XyzGateway` implements `PaymentGateway` no pacote `infra.payment`. Anotar com `@Component("xyz")`.

2

Implementar os 4 métodos

`createPayment`, `getStatus`, `refund` e `processWebhook`. Mapear modelos do provider para os DTOs internos.

3

Adicionar variáveis de ambiente

Credenciais do novo gateway no `application.yml` e documentar na tabela de env vars do backend.

4

Registrar endpoint de webhook

O `PaymentController` roteia para o gateway correto via provider. Nenhuma mudança no controller — apenas configurar a URL no painel do provider.

5

Testes de integração

Implementar `XyzGatewayIntegrationTest` com mocks do provider SDK. Testar todos os cenários de webhook.

mapa de gateways futuros

Provider	Mercado Pago	Stripe	PagarMe	PayPal
Status	Atual	Planejado	Planejado	A avaliar
Pix	✓	—	✓	—
Cartão de Crédito	✓	✓	✓	✓
Boleto	✓	—	✓	—
Recorrência	—	✓	—	—

paymentrequest dto

DTO de entrada para `PaymentGateway.createPayment()`. Agnóstico ao provider.

Campo	Tipo	Obrigatório	Descrição
<code>orderId</code>	String	☒	ID interno do pedido para correlação
<code>amount</code>	BigDecimal	☒	Valor em reais (ex: <code>149.90</code>)
<code>method</code>	PaymentMethod	☒	Enum: <code>PIX</code> , <code>CREDIT_CARD</code> , <code>BOLETO</code>
<code>customer</code>	CustomerDTO	☒	Nome, e-mail, CPF do comprador
<code>items</code>	List<ItemDTO>	☒	Itens do pedido com título, qty e unit price
<code>cardToken</code>	String	☐	Token do cartão (gerado pelo SDK JS do gateway)
<code>installments</code>	Integer	☐	Número de parcelas. Padrão: <code>1</code>
<code>metadata</code>	Map<String,String>	☐	Dados extras para rastreamento (ex: UTM source)